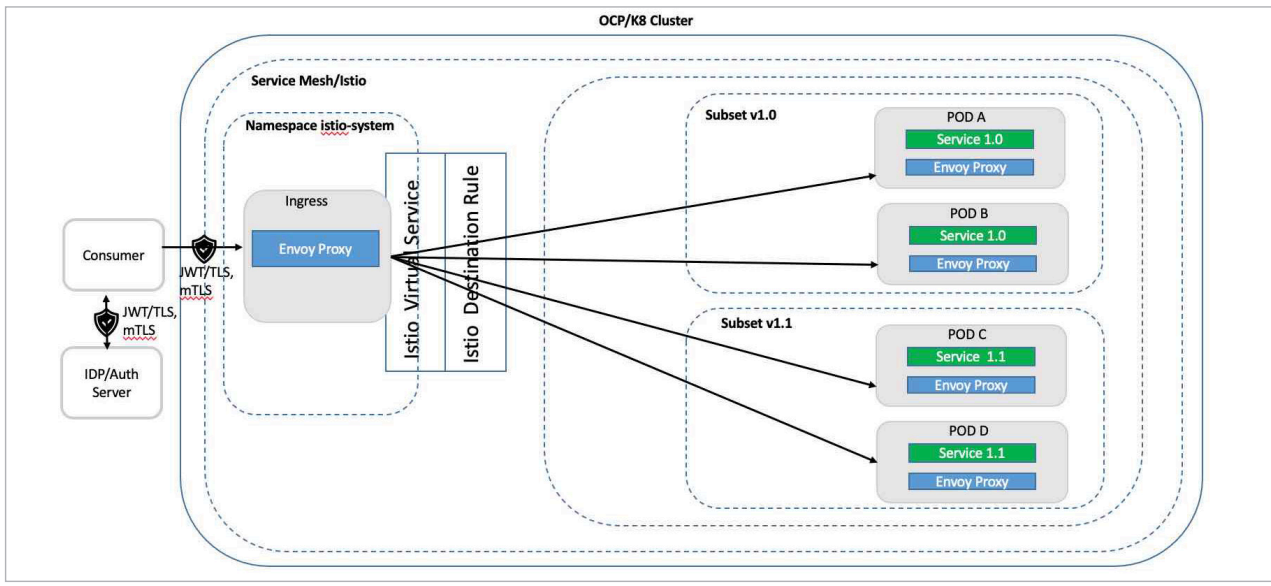

Figure 7: Service/container deployment

minor version 1.1 is introduced, and it is decided that 10 per cent of the traffic should be routed to the new version to confirm it is behaving properly before routing all the traffic to it. As this is a minor version change, there should not be any change message schema for the consumer. By deploying the new version of the service in production and routing 10 per cent of the traffic to it through *istio* config, deployment of new code is separated from the cutover and validates that all is correct at each step before increasing the proportion of the traffic to the new version. Once all the traffic is routed to version 1.1, version 1.0 can be decommissioned.

Dark launches: Dark launches or A/B testing allows new versions of the service to be deployed in production separately from cutover activity. It is different from canary release in that a specific number of early adopters are moved to the new functionality to use it before it is rolled out fully. An *http* header is used to determine whether to route the request to the new version of the service. Istio can check the *http* header in the virtual service to get the name of the consuming application or some other flag, and route it to the new version. For an external consumer, an alternative approach could be to use subject alternative name (SAN) from the x.509 certificate to identify the consumer, as these details are propagated by the ingress gateway in an *http* header.

Traffic mirroring: Traffic mirroring or shadow launch with service mesh provides a way of deploying new services in a production environment and then sending production traffic to the new service out of band. It helps in understanding how the new service is performing with production traffic in comparison to the currently live service. As the new service is out of band, it is isolated from the existing old service and any issue or error in the new service will not impact the current service or the consumer. Care must be taken that out of band traffic does not manipulate state in live data stores. The primary use cases for this will be read-only transactions.

Figure 7 shows an application in canary deployment, dark launches and traffic mirroring where a new version v1.1 is being introduced and v1.0 is currently in use. Note that minor versions in the production environment must only be used for the transition period, and the old minor version should sunset as soon as the new minor version is stable. There are two replicas in the example of service 1.0 and 1.1 in Figure 7.

Impact of the pattern

Implementing these deployment patterns in any project should allow for more than one change deployment slot as there will be updates to the routing rules after the initial release, as well as removal of old services and pods at the end of the cycle to avoid proliferation of live services with multiple minor versions. Also, consumers should be given time for upgrading/refactoring in code, before the removal of previous versions of services from production. An alternate approach is to implement a gateway service pattern to minimise the impact on consumers.

From a technical design standpoint, a suitable regex will need to be developed and maintained for querying x509 certificate to identify the consumer, and similarly querying JWT in the *http* header for specific claims. END

By: Pradip Roychowdhury

The author is a distinguished chief technologist with 25 years of experience in areas of OOP, SOA, cloud, DevOps and banking transformation. He is The Open Group certified 'Distinguished Architect'.